

# Stage 1 Lab - Human vs AI: Building Your First SmartCare Prototype

---

## Learning objectives

- Create and run a simple Python file with basic input,output and processing statements
- Use lists, dictionaries and functions to enhance the Python file
- Build a small SmartCare appointment prototype.
- Use AI as a tutor rather than a replacement.
- Compare human-written and AI-generated code.
- Verify AI-generated code through execution and test inputs.
- Document a short AI-use reflection.

## Files to create and commit in GitHub

```
stage01/  
  smartcare_v01.py  
  comparison.md  
  reflection.md  
  ai_usage.md
```

**\*\***Isn't the above already shown in this doc? Except for .py file which is in the folder but also in this doc

## Part A - Understand the Problem: AI OFF

SmartCare needs a small prototype that allows a receptionist to record patient appointments. Each appointment records patient name, practitioner name and appointment time.

What data must be stored?

- Patient name
- Practitioner name
- And Appointment time/date

What functions might be useful?

- Can add a booking -> add\_appointment
- all\_appointments
- check\_availability
- record\_details
- user\_profile

What could go wrong?

- Wrong inputs, incorrect/misleading details/information, adjusting time for region, managing between branches, accidental duplicate bookings, manual add appointment vs patient registering and making appointment (can lead to duplicate bookings), changing/managing patient details and/or appointments, no-show appointments, assigning practitioners to patients

What requirements are unclear?

- Are there multiple branches?
- Practitioners by branch (if more than 1)
- Appointment by need/appointment type (medical need etc)

## Part B - Build a Human-Written Prototype: AI OFF

### #task 1

# Create and run a simple Python file with basic input/output statements

```
print("Welcome to SmartCare: Community Clinic Appointment Booking System!")
```

# First Appointment

```
patient1_name = 'Alice Smith'
```

```
practitioner1_name = 'Dr. John Doe'
```

```
appointment1_time = '2024-07-20 10:00 AM'
```

```
print(f"Patient: {patient1_name} | Practitioner: {practitioner1_name} | Time: {appointment1_time}")
```

# Second Appointment

```
patient2_name = 'Bob Johnson'
```

```
practitioner2_name = 'Dr. Jane Roe'
```

```
appointment2_time = '2024-07-20 11:30 AM'
```

```
print(f"Patient: {patient2_name} | Practitioner: {practitioner2_name} | Time: {appointment2_time}")
```

### #task1enhanced

# Use lists, dictionaries and functions to enhance the Python file

```
appointments = []
```

```
def book_appointment(patient_name, practitioner_name, appointment_time):
```

```
    if not patient_name:
```

```
        raise ValueError("Patient name cannot be empty")
```

```
    appointment = {
```

```
        "patient": patient_name,
```

```
        "practitioner": practitioner_name,
```

```
        "time": appointment_time
```

```
    }
```

```

appointments.append(appointment)

def display_appointments():
    if not appointments:
        print("No appointments recorded.")
        return
    for appointment in appointments:
        print(f"Patient: {appointment['patient']} | Practitioner:
{appointment['practitioner']} | Time: {appointment['time']}")

print("Welcome to SmartCare: The Clinical Appointment Booking System!")
book_appointment('Alice Smith', 'Dr. John Doe', '2024-07-20 10:00 AM')
book_appointment('Bob Johnson', 'Dr. Jane Roe', '2024-07-20 11:30 AM')
display_appointments()

```

^ Now, run both programs , and identify at least five limitations.

5 limitations:

1. Both codes don't require user input (doesn't distinguish between SmartCare staff or patient input if patients made appointments themselves)
2. No room for checking invalid inputs
3. No appointment edit option
4. Appointment classification not there
5. ValueError specifies that patient name cannot be empty, but practitioner and appointment date/time can be? Also this points out that although patient name cannot be empty, there is no way of deciding the output of that as there is no way to enter any type of input.

## Part C - Use AI as Tutor: AI ON (Use only UC approved GenAI Tool such as Microsoft CoPilot)

Suggested prompt structure:

Act as a Python tutor.

I am learning introductory software technology.

Here is a small appointment-booking function.

1. Explain what the code does.

The program prints a welcome message, then displays two hard-coded appointments using simple variables. In the enhanced section, it introduces a list (appointments) and two functions:

- `book_appointment()` — creates a dictionary containing patient, practitioner and time, then adds it to the list. It also checks that the patient name is not empty.
- `display_appointments()` — prints all stored appointments, or a message if none exist.

Finally, it books two appointments and displays them.

2. Identify three limitations.

- **No conflict checking** — the system allows two appointments at the same time with the same practitioner.
- **Weak validation** — only the patient name is checked; practitioner and time could be blank or None.
- **No user input** — everything is hard-coded, so the receptionist cannot actually enter appointments.

3. Suggest improvements.

- Add validation for **practitioner name** and **appointment time** to prevent empty or None values.
- Add a simple **conflict check** inside `book_appointment()` to warn if a practitioner already has an appointment at that time.
- Add a small helper function like **validate\_time** to ensure the time format is consistent.

4. Do not rewrite the whole application.

5. Ask me two questions to test my understanding.

1. Why is a **list of dictionaries** more flexible than using separate variables like `patient1_name`, `patient2_name`, etc.?
2. What happens in your program if you call `book_appointment(None, 'Dr. John Doe', '10:00 AM')`?

\*\*\*Microsoft Copilot used as instructed

## Part D - Generate an Alternative: AI ON

Ask AI to create a simple beginner-friendly Python function that stores patient name, practitioner name and appointment time. Explicitly prohibit a database or GUI.

```
def create_appointment(patient_name, practitioner_name, appointment_time):
```

```
    # Basic validation to keep things beginner-friendly
```

```
    if not patient_name:
```

```
        raise ValueError("Patient name cannot be empty.")
```

```
    if not practitioner_name:
```

```
        raise ValueError("Practitioner name cannot be empty.")
```

```
    if not appointment_time:
```

```
        raise ValueError("Appointment time cannot be empty.")
```

```
# Store the appointment in a simple dictionary (no database, no GUI)
```

```
appointment = {  
    "patient": patient_name,  
    "practitioner": practitioner_name,  
    "time": appointment_time  
}
```

```
return appointment
```

```
# Example usage
```

```
appt = create_appointment("Alice Smith", "Dr. John Doe", "2024-07-20 10:00 AM")
```

```
print(appt)
```

```
***Microsoft Copilot used as instructed
```

## Part E - Compare Human and AI Versions

| Question                     | Human version | AI version |
|------------------------------|---------------|------------|
| Easy to understand?          | Yes           | Yes        |
| Runs successfully?           | Yes           | Yes        |
| Uses only required features? | Yes           | Yes        |
| Adds assumptions?            | No            | Yes        |
| Handles errors?              | No            | Yes        |
| Could I explain it?          | Yes           | Yes        |

## Part F - Verify Behaviour

- Normal appointment --> Okay
- Blank patient name --> Does not handle
- Two appointments for the same practitioner/time --> Does not handle
- Strange input such as patient\_name=None or appointment\_time=None --> Does not handle

## Part G - Improve One Thing

Choose exactly one controlled improvement, for example: if not patient\_name: raise ValueError("Patient name cannot be empty")

- I would improve/add on that by raising ValueError for appointment time and practitioner as those are not specifically addressed

## Part H - Reflection (150-250 words)

What did you build before using AI?

- A basic appointment booking system where you manually enter values as rows of code in the system/application. Covering bare details like time/date, patient name, and practitioner; nothing more.

What did AI help you understand?

- AI helped me understand that error handling like for practitioner and appt time can be written under def can improve the function of the code and that dictionaries are structurally better in terms of organizing the code; making it easier to read and understand

Did AI make assumptions?

- Yes, like handling errors for practitioner name and appt time being blanks.

How did you verify the AI output?

- On VS Code

What engineering work remained for you?

- Verifying code readability and function

## Submission checklist [GitHub Commit]

- Python file runs.
- Comparison table completed.
- Normal and unusual inputs tested.
- AI assistance documented.
- Reflection completed.
- I can explain my code.